

Control PID

Miguel Casallas

Definición

El control PID (Proporcional-Integral-Derivativo) consiste en un sistema paralelo que permite manejar el error obtenido entre la referencia (setpoint) y una variable medida a través de un sensor.

$$e(t) = sp - y(t)$$

Dicho error se maneja en tres operaciones para calcular la acción de control ($u(t)$) necesaria, buscando corregir este error.

$$u(t) = Kp * e(t) + Ki \int_0^t e(\tau) + Kd * \frac{de(t)}{dt}$$

En donde $Kp * e(t)$ trabaja el error instantáneo, $Ki \int_0^t e(\tau)$ trabaja el error pasado y $Kd * \frac{de(t)}{dt}$ se basa en el error futuro a través de predicciones.

Aplicación en VEX

Ejemplo 1 Sensor de Inercia para llegar a un ángulo deseado.

Objetivo: Hacer que el robot gire a un ángulo deseado usando la IMU (Unidad de medida inercial), controlando los motores con PID para que se detenga justo ahí, sin pasarse ni oscilar.

1. Crear una estructura donde se guardan los datos del PID.

```
struct PID {  
    double kp = 0;  
    double ki = 0;  
    double kd = 0;  
  
    double error = 0;  
    double prevError = 0;  
    double integral = 0;  
};
```

- kp, ki, kd : son los parámetros del controlador. Se ajustan según se observe que el sistema oscila o no realiza el movimiento indicado.
 - $error$: diferencia entre el ángulo que queremos y el que tenemos.
 - $prevError$: el error del ciclo anterior, para calcular la derivada.
 - $integral$: suma acumulada de errores.
2. Definir una función que calcula la salida PID
La función usa la formula PID para calcular cuanto se deben mover los motores.

```
double computePID(PID &pid, double setpoint, double current, double dt) {
    pid.error = setpoint - current;
    pid.integral += pid.error * dt;
    double derivative = (pid.error - pid.prevError) / dt;

    double output = pid.kp * pid.error + pid.ki * pid.integral + pid.kd * derivative;

    pid.prevError = pid.error;
    return output;
}
```

- setpoint: valor objetivo.
- current: valor actual leído por el sensor.
- dt: tiempo entre dos mediciones, en segundos.
- $kp * \text{error}$: corrige según qué tan lejos estás del objetivo (Control Proporcional).
- $ki * \text{integral}$: suma de errores anteriores, para eliminar error final.
- $kd * \text{derivative}$: corrige según qué tan rápido cambia el error, para evitar sobrepasarse.

3. Configurar Motores y Sensores.

```
motor leftMotor(PORT1, ratio18_1, false);
motor rightMotor(PORT2, ratio18_1, true);
inertial imuSensor(PORT3); // Conectado al puerto 3

brain Brain;
```

4. Configuración del Main

```
int main() {
    vexcodeInit();

    // Calibrar el IMU (espera hasta que termine)
    imuSensor.calibrate();
    while (imuSensor.isCalibrating()) {
        wait(20, msec);
    }

    // Crear e inicializar el controlador PID
    PID pid;
    pid.kp = 2.0;    // Respuesta rápida
    pid.ki = 0.01;   // Elimina error final (pequeño)
    pid.kd = 0.3;    // Amortigua (evita que se pase)

    double target = 90.0; // Queremos girar a 90 grados

    timer t;
    double lastTime = t.time(msec); // Tiempo inicial

    while (true) {
        double currentTime = t.time(msec);
        double dt = (currentTime - lastTime) / 1000.0; // en segundos
        lastTime = currentTime;

        double angle = imuSensor.rotation(degrees); // Lectura actual del IMU

        double power = computePID(pid, target, angle, dt);

        // Limitar la potencia entre -100 y 100
        if (power > 100) power = 100;
        if (power < -100) power = -100;
```

```
// Girar: un motor hacia adelante y el otro hacia atrás
leftMotor.spin(forward, power, percent);
rightMotor.spin(forward, -power, percent);

// Esperar antes del próximo ciclo
wait(20, msec);
}
```

- **imuSensor.calibrate();**
Empieza la **calibración del sensor IMU**. Esto es importante para que las lecturas del ángulo estén correctas. Si no se calibras, el sensor puede dar valores incorrectos.
- **while (imuSensor.isCalibrating()) {**
 wait(20, msec);
 }
Espera hasta que la IMU termine de calibrarse. El programa no sigue hasta que el sensor esté listo.
Pone una pausa de 20 milisegundos entre cada verificación para no saturar la CPU.
- **pid.kp = 2.0;**
- **pid.ki = 0.01;**
- **pid.kd = 0.3;**
Asignan los valores del controlador:
kp = 2.0: sensibilidad proporcional.
ki = 0.01: para corregir errores acumulados.
kd = 0.3: para amortiguar (evitar que se pase del objetivo).
- **double target = 90.0;**
Define el ángulo deseado.
- **timer t;**

double lastTime = t.time(msec);

Crea un temporizador para medir el tiempo entre cada iteración del bucle (dt, delta de tiempo).
lastTime guarda el tiempo en milisegundos del ciclo anterior.
- **double currentTime = t.time(msec);**

double dt = (currentTime - lastTime) / 1000.0;

lastTime = currentTime;

Calcula el tiempo transcurrido (dt) entre ciclos.

Se mide en milisegundos y se divide entre 1000 para convertirlo a segundos, que es la unidad que usa el PID.

Luego se actualiza lastTime para el próximo ciclo.

- **double angle = imuSensor.rotation(degrees);**
Lee la posición angular actual del robot desde el sensor IMU.
- **double power = computePID(pid, target, angle, dt);**

Calcular la salida del PID.

Esta función toma:

El pid (con sus parámetros), El target (ángulo deseado), El angle (ángulo actual), El dt (tiempo entre ciclos).

Y devuelve un número que representa cuánta potencia aplicar a los motores para corregir el error.

- **if (power > 100) power = 100;**

if (power < -100) power = -100;

Limita la salida entre -100 % y +100 % de potencia.

Así evitamos que el motor reciba órdenes demasiado altas. (Para nuestro caso se debe ajustar al porcentaje que reciben los motores como parámetro o la velocidad).

- **leftMotor.spin(forward, power, percent);**
rightMotor.spin(forward, -power, percent);

Aplicar la potencia a los motores:

El motor izquierdo gira hacia adelante con power.

El derecho gira al revés con -power para que el robot gire sobre su eje.

¿Cómo ajustar el PID?

1. Comienza con solo proporcional:
 - Poner $k_i = 0$, $k_d = 0$.
 - Subir k_p hasta que el robot llegue rápido, pero sin oscilar mucho.
2. Añadir derivativa (k_d) si el robot se pasa o vibra.
3. Añadir integral (k_i) si el robot se queda corto o nunca llega exacto.

¿Qué hace este código físicamente?

Imaginar que el robot está sobre el suelo, y se quiere que gire exactamente 90 grados a la derecha. Para lograrlo con suavidad y precisión, se usa el sensor IMU, que mide el ángulo actual del robot en tiempo real.

Entonces el robot compara constantemente:

- **Dónde está** (ángulo actual del IMU)
- **A dónde quiere llegar** (ángulo objetivo: 90°)

- **Cuánto le falta** (error = objetivo - actual)

Ese “cuánto le falta” es el **error**, y el robot lo usa para decidir:

- ¿Debo girar más rápido?
- ¿Debo frenar?
- ¿Estoy a punto de pasarme?

El controlador PID hace esto:

1. Proporcional (P) – "¿Qué tan lejos estoy?"

- Si el robot está muy lejos del ángulo objetivo, gira más rápido.
- Si está cerca, gira más despacio.

2. Integral (I) – "¿He estado mucho tiempo con error?"

- Suma los errores que no ha podido corregir completamente.
- Aporta un empuje extra si el robot se queda corto.

3. Derivativo (D) – "¿Estoy girando muy rápido?"

- Observa qué tan rápido se está corrigiendo el error.
- Si va demasiado rápido, **frena** para que no se pase.